TASK 2 – USE TOKENS, KEYWORDS, IDENTIFIERS AND CONSTANTS CORRECTLY

2.1 Definition of Constants

Constants are fixed values that cannot be changed during program execution. In the metro fare system, fares, surcharge percentage and minimum card balance should be declared centrally to avoid repeated values and inconsistent updates. The following C++ declarations define the required values:

```
constexpr int INNER_FARE_PAISE = 1000;
constexpr int MIDDLE_FARE_PAISE = 2000;
constexpr int OUTER_FARE_PAISE = 3000;

constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
```

The fares are stored in paise to represent money accurately. For example, Rs. 10 is represented as 1000 paise.

The concession percentages for students and senior citizens are not specified in the assignment scenario; therefore, they must be defined only when the actual rates are provided.

2.2 Five Kinds of C++ Tokens

Token Type	**Meaning**	**Examples from the Metro System**
Keywords	Reserved words with predefined meanings in C++	int, class, if, return, const
Identifiers	Names given to variables, functions and classes	INNER_FARE_PAISE, SmartCard, calculateFare()

Constants	Fixed values that do not change	1000, 2000, 20, 5000
Operators	Symbols used to perform operations	+, -, *, >=, ==
Punctuators	Symbols used to organise program statements	{ }, ( ), ;, ,

2.3 Comparison of #define, const and constexpr

1. #define

The #define directive creates a preprocessor macro. It replaces the macro name with its specified value before compilation.

Example:

```
#define INNER_FARE 10
#define PEAK_SURCHARGE 20
```

It does not provide normal C++ type checking or variable scope.

2. const

The const keyword declares a variable whose value cannot be modified after initialisation.

Example:

```
const int MINIMUM_BALANCE = 50;
```

It provides type checking and follows normal C++ scope rules.

3. constexpr

The constexpr keyword declares a value that can be evaluated at compile time when its initialisation satisfies constant-expression requirements.

Example:

constexpr int INNER_FARE_PAISE = 1000;
constexpr int MIDDLE_FARE_PAISE = 2000;
constexpr int OUTER_FARE_PAISE = 3000;

It is suitable for fixed fare values because the values are known before program execution.

Comparison:

Feature	**#define**	**const**	**constexpr**
Processing	Preprocessor replacement	C++ declaration	C++ declaration
Type checking	No normal type checking	Yes	Yes
Scope	Not governed by normal C++ scope	Normal C++ scope	Normal C++ scope
Modification	Macro replacement is textual	Cannot be modified	Cannot be modified
Compile-time value	Textual substitution	Not necessarily	Yes, when valid
Suitability	Less suitable for fare data	Suitable for fixed values	Preferred for fixed fare values

Justification: constexpr is preferred for the metro fare table because the fare values and surcharge percentage are fixed constants. It provides type safety, clear declarations and compile-time evaluation.

2.4 enum and enum class for Metro Zones

An enumeration represents a fixed set of named values.

Traditional enum:

enum Zone

```
{
   Inner,
   Middle,
   Outer
};
```

A traditional enum allows its enumerator names to be used in the surrounding scope and can be implicitly converted to an integer.

Scoped enum (enum class):

```
enum class Zone
{
   Inner,
   Middle,
   Outer
};
```

The values are accessed using the scope resolution operator:

```
Zone::Inner
Zone::Middle
Zone::Outer
```

Comparison:

Feature	**enum**	**enum class**
Scope	Enumerator names can enter the surrounding scope	Enumerator names remain inside the enum scope
Type safety	Less restrictive	Stronger type safety
Integer conversion	Implicit conversion is allowed	Explicit conversion is required

Name conflicts	More likely	Less likely
Suitability	Less suitable for larger systems	Preferred for metro zones

Justification: enum class is safer because it prevents accidental mixing of zone values with unrelated integer values and avoids naming conflicts. Therefore, it is preferred for representing Inner, Middle and Outer zones.

2.5 Identifier Naming Rules and Conventions

Identifiers are names assigned to program elements such as variables, functions and classes.

The following naming rules and conventions should be followed:

1. An identifier must begin with a letter or underscore, not a digit.
2. It can contain letters, digits and underscores.
3. Spaces and special characters are not allowed.
4. C++ keywords cannot be used as identifiers.
5. Identifiers are case-sensitive.
6. Meaningful names should be used to improve readability.

Examples:

Identifier	**Purpose**	**Validity**
INNER_FARE_PAISE	Inner-zone fare	Valid
MIN_BALANCE_PAISE	Minimum balance	Valid
calculateFare	Fare calculation function	Valid
SmartCard	Smart-card class	Valid
2fare	Starts with a digit	Invalid
class	Reserved C++ keyword	Invalid

fare amount	Contains a space	Invalid

Using uppercase letters for constants, PascalCase for class names and camelCase for functions makes the program easier to understand and maintain.

2.6 Alternative Concept and Complexity Analysis – Using #define Instead of constexpr and enum class

If the metro fare system used #define macros instead of constexpr constants and enum class, fare values would be substituted by the preprocessor, and zones could be represented using macro-defined numbers.

Example:

```
#define INNER_FARE_PAISE 1000
#define MIDDLE_FARE_PAISE 2000
#define OUTER_FARE_PAISE 3000
#define PEAK_SURCHARGE_PERCENT 20
#define MIN_BALANCE_PAISE 5000

#define INNER 0
#define MIDDLE 1
#define OUTER 2
```

This approach is shorter, but it does not provide the same type safety and scope protection as constexpr and enum class.

Complexity Comparison

Factor	**#define Approach**	**constexpr and enum class Approach**
Type checking	No normal type checking for macros	Stronger type checking

Scope	No normal C++ scope	Follows C++ scope rules
Memory	Macro itself occupies no storage	A constant usually requires no separate storage when used as a compile-time value
Running time	No additional runtime cost for macro substitution	No additional runtime cost for compile-time constants
Error prevention	More risk of accidental misuse	Better protection against type and naming errors
Maintainability	Less structured	More structured and easier to maintain

Time Complexity: Both approaches have O(1) time complexity for accessing a fixed fare value. Macro substitution occurs before compilation, while compile-time constants can be evaluated during compilation.

Space Complexity: A macro itself occupies no storage. A constexpr constant usually requires no separate storage either, although storage may be required if its address is used or an object is otherwise required.

Conclusion: The constexpr and enum class approach is more suitable because it provides better type safety, scope control and maintainability without introducing additional runtime cost for these fixed values.

2.7 Conclusion

The metro fare system should use meaningful identifiers, centrally declared constexpr constants and enum class for zone representation. These choices reduce repeated fare values, prevent naming conflicts and improve type safety. Compared with #define macros and traditional enums, the recommended approach provides a more reliable and maintainable C++ design.